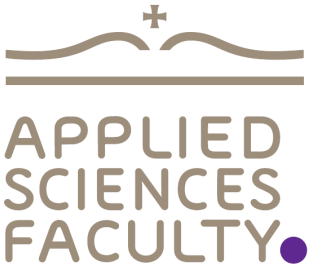
C++ — лекція 22. Ітератори





©Олег Фаренюк,
Creative Commons - Attribution Share Alike (CC BY-SA)

Ітератори

- **Ітератори** — узагальнення поняття вказівника.
- Для вбудованого масиву C ітератором є вказівник на його елемент. Для vector, string та інших контейнерів використовується спеціалізовані типи ітераторів, які, в залежності від реалізації (та режиму компіляції[1]), можуть бути як вказівниками так і спеціальними класами, що перевантажують відповідні оператори.
- Ітератор вказує на елемент контейнера. Тобто, цитуючи Страуструпа, ітератор абстрагує поняття вказівника на елемент послідовності.
- Для деяких контейнерів, таких як vector, deque та string, є можливість працювати із елементами, використовуючи індекси, однак, для багатьох контейнерів індексів не існує (list, set, map і т.д.), тому ітератори є єдиним способом працювати із ними.

[1] Наприклад, в режимі відладки це клас із перевіркою виходу за межі контейнера, а в режимі релізу — вказівник.

Згідно популярного в сучасному C++ принципу, "все, що виглядає як качка, літає як качка, крякає як качка, певне, є качкою", ітератором є будь-яка сутність, котра реалізовує наступні операції:

- Доступ до елемента, на який він вказує — оператори * і ->
- Можливість перейти до наступного елемента послідовності — оператор ++.
- Можливість порівняння, оператор ==.

Вище перераховано мінімальний необхідний набір властивостей ітераторів. Ітератори, якщо це дозволяє "підлеглий" контейнер, також можуть надавати додаткові можливості — рух назад (оператор --), довільні переміщення (оператори + та -), тощо.

Ітератори стандартних контейнерів

- Типи ітераторів для стандартних контейнерів оголошені в самих контейнерах, з іменами iterator, const_iterator для константного ітератора, reverse_iterator та const_reverse_iterator:

```
vector<int>:: iterator   itr ;
```

- Стандартні контейнери, такі як vector, надають ряд методів роботи з ітераторами, зокрема, отримати ітератор, що вказує на перший елемент контейнера, і ітератор, що вказує на елемент після останнього: begin() та end().
- Наприклад, проходимо по всіх елементах vector так:

```
vector<int> arr1;
                .....
for(vector<int>:: iterator   i  = arr1.begin();
          i  != arr1.end();  ++i ){
     cout  << *i << " ";
}
```

В залежності від можливостей ітераторів, їх ділять на декілька категорій[2].

Категорія	Властивості	Приклад
Всі ітератори	Копіювання Інкремент	`X a(b);` `++a; a++;`
Виводу (output)	Розіменування як lvalue Перевірка рівності ==, та не рівності !=	`*a = x` `a==b; a!=b;`
Вводу (input)	Розіменування як rvalue	`t = *a;` `t = a->m;`

Однонаправлені (forward)	Все, що і ітератори вводу та ітератори виводу
	Конструктор по замовчуванню
	Багатопрохідні [3]
		
		`X a;`
		`b=a; *a++; *b;`

Двонаправлені (bidirectional)	Все, що і однонаправлені ітератори Декремент: `--`	 `--a; a--;`

Довільного доступу (random access)	Все, що і двонаправлені
	Арифметичні операції, +, -
	
	Оператори відношення: >, >=, <, <=
	
	Оператор звертання за індексом (за зміщенням)
		
		`b=a+n; b=a-n;`
		`b=n+a;b=n-a;`
		`a>b; b>a; a>=b;`
		`b>=a;`
		`x = a[n];`

[2] Див. також таблицю тут `http://www.cplusplus.com/reference/iterator/`

[3] Пересування та розіменування не впливає на можливість розіменування. Зокрема, дві копії, над якими виконували одні і ті ж операції, відповідатимуть тому ж елементу

<iterator> — допоміжні засоби роботи з ітераторами

- Щоб знайти віддаль між двома ітераторами — скільки між ними елементів, варто використовувати функцію `distance(itr1, itr2)`. В залежності від типу ітератора, вона вибере найоптимальніший спосіб обчислення віддалі — для ітераторів довільного доступу скористається оператором -, для інших виконуватиме ++ до itr1, поки не досягне itr2. Тобто, itr2 має бути *досяжним* таким способом із itr1.
- Аналогічно, щоб перемістити ітератор на n елементів, варто використовувати `advance(itr,n)`. Функція може отримувати як додатні, так і від'ємні n.
- Ітератором може бути будь-який тип, який задовольняє описаним раніше вимогам, однак, для спрощення роботи із ними — можливості написання загального коду, стандартна бібліотека надає два допоміжних засоби, шаблон iterator_traits та його спеціалізації, що описують властивості ітераторів, і шаблон iterator, для спрощення написання своїх спеціалізацій для iterator_traits.
- Крім того, в цьому ж файлі заголовків знаходяться оголошення адаптерів reverse_iterator, back_insert_iterator, front_insert_iterator, insert_iterator, ітераторів потоків, про які у лекції про потоки, та відповідних генераторів, back_inserter, front_inserter, inserter.

<iterator> — C++11

C++11 додала корисні незалежні функції:

- begin() — для контейнерів, котрі мають метод begin(), повертає його результат, для вбудованих масивів повертає вказівник на початок.
- end() — для контейнерів повертає результат їх методу end(), для масивів — arr+N.
- next(it, n) — повертає ітератор, на який буде вказувати it, якщо його зсунути на n.
- prev(it, n) — аналогічно, але на -n позицій.

Також додано новий адаптер move_iterator і відповідну функцію-генератор make_move_iterator. При розіменуванні такий ітератор повертає rvalue reference, як після використання std::move).

Шаблон std::iterator_traits<>

- Загальний шаблон:

```
template <class T>
struct  iterator_traits  {
    typedef typename T::value_type          value_type;
    typedef typename T::difference_type     difference_type ;
    typedef typename T::iterator_category   iterator_category ;
    typedef typename T::pointer             pointer ;
    typedef typename T::reference           reference ;
};
```

Тобто, він розрахований на роботу із класами ітераторів, котрі мають поля типів value_type і т.д.

- Звичайні вказівники таких полів, природно, не мають. Для них є спеціалізація:

```
template <class T>
struct  iterator_traits <T*> {
    typedef T                           value_type;
    typedef  ptrdiff_t                  difference_type ;
    typedef random_access_iterator_tag  iterator_category;
    typedef T*                          pointer ;
    typedef T&                          reference ;
};
```

- Цей клас оголошено застарілим в C++17.
- відповідні поля потрібно оголосити вручну – виграшу від використання такого успадкування не було.
- Для своїх ітераторів можна також писати відповідні спеціалізації iterator_traits.
- Щоб спростити цей процес, в <iterator> є простий клас iterator:

```
template <class Category, class T, class Distance = ptrdiff_t,
          class Pointer = T*, class Reference = T&>
  struct iterator {
    typedef T         value_type;
    typedef Distance  difference_type;
    typedef Pointer   pointer;
    typedef Reference reference;
    typedef Category  iterator_category;
  };
```

Успадкування від нього дозволяє легко оголосити всі поля, необхідні iterator_traits, за потреби, налаштувавши частину із них під свої потреби.

```
class MyIntFwdIterator
    : public std :: iterator <std::forward_iterator_tag, int>{
  int* p;
public:
  MyIntFwdIterator() :p(0) {}
   explicit MyIntFwdIterator(int* x) :p(x) {}
MyIntFwdIterator(const MyIntFwdIterator& mit) : p(mit.p) {}
  MyIntFwdIterator& operator++() {++p;return *this;}
  MyIntFwdIterator operator++(int)
      {MyIntFwdIterator tmp(*this); operator++(); return tmp;}
  bool operator==(const MyIntFwdIterator& rhs) const
                                          {return p==rhs.p;}
  bool operator!=(const MyIntFwdIterator& rhs) const
                                          {return p!=rhs.p;}
  int& operator*() const {return *p;}
};
  .........
const size_t N=20;
int arr[N];
MyIntFwdIterator arr_itr_begin(arr), arr_end(arr+N);
for(MyIntFwdIterator arr_itr = arr_itr_begin; arr_itr!=arr_end; ++arr_itr)
  *arr_itr = 5;
```

iterator_category

- В iterator_traits та iterator фігурує поле iterator_category. Розглянемо його.
- Не зважаючи на те, що категорії ітераторів явно відповідають поняттю успадкування — ітератор кожної наступної категорії вміє все, що попередня категорія, плюс щось своє (вважаючи, що forward ітератор успадковує одночасно і від input і від output), будувати із них єдину ієрархію успадкування незручно! Різні класи ітераторів цілком можуть бути ніяк більше, крім їх категорії, не бути пов'язаними.
- Тому стандартна бібліотека пішла дещо іншим шляхом. Існує "паралельна" ієрархія категорій ітераторів:

```
struct input_iterator_tag { };
struct output_iterator_tag { };
struct forward_iterator_tag : public input_iterator_tag { };
struct bidirectional_iterator_tag : public forward_iterator_tag { };
struct random_access_iterator_tag : public bidirectional_iterator_tag
                                                                { };
```

котра дозволяє вибирати алгоритми, під час компіляції, поліморфно, в залежності від категорії, але на заганяючи ітератори в єдину ієрархію.

- Розглянемо підхід вибору алгоритму, у залежності від категорії ітераторів на прикладі функції distance.
- Загальний код, який, власне, і викликатиме користувач:

```
template <typename Iterator>
typename std:: iterator_traits <Iterator>:: difference_type
distance ( Iterator pos1, Iterator pos2){
  return distance (pos1, pos2,
                   std :: iterator_traits <Iterator>
                          :: iterator_category ());
}
```

Зауважте, як тип результату визначається із допомогою iterator_traits, звідтіля ж береться і тип-тег категорії ітераторів.

- Варіант для ітераторів довільного доступу:

```
template <typename RandomAccessIterator>
typename std:: iterator_traits <RandomAccessIterator>::difference_type
distance (RandomAccessIterator first_position ,
          RandomAccessIterator second_position,
                std :: random_access_iterator_tag){
  return second_position - first_position ;
}
```

Вибір варіанту здійснюється завдяки перевантаженню!

- Варіант для ітераторів вводу та потужніших:

```
template <typename InputIterator>
typename std:: iterator_traits <InputIterator >:: difference_type
distance ( InputIterator first_position , InputIterator second_position,
                    std :: input_iterator_tag) {
  typename std:: iterator_traits <InputIterator >:: difference_type diff ;
  for ( diff =0; first_position != second_position;
        ++first_position, ++diff) {}
  return diff ;
}
```

  Вибір варіанту здійснюється завдяки перевантаженню! При чому, завдяки тому, що теги категорій утворюють ієрархію успадкувань, не потрібно окремо писати такі функції для нащадків input_iterator_tag.

- Звичайно, цей же трюк можна використовувати для написання власних функцій, чия поведінка має залежати від категорії ітераторів.

- Приклад створення ітератора, із використанням C++20: "An iterator_facade in C++20"
- "Simple C++20 input and output iterators", універсальний клас ітератора, який отримує чи відправляє дані за допомогою переданої функції.
- Ще один приклад створення ітераторів: "Custom Iterator" та "A neat technique for custom output iterators".
- Серія статей від Раймонда Чена:
  - "How do I create an inserter iterator that does unhinted insertion into an associative container like std::map?".
  - "Creating a generic insertion iterator, part 1".
  - "Creating a generic insertion iterator, part 2".
- Трохи загальніший есей про два види тегів у стандартній бібліотеці: "Two kinds of tag types: foo_t and foo_tag" – для вибору перевантаження конструкторів (типу std::defer_lock_t для RAII-обгорток мютексів) та для шаблонного метапрограмування (як вище, з ітераторами).

